

Transcription automatique audio guitare → MIDI

Soutenance pour le titre professionnel *Concepteur Développeur en Science des Données*

Damien DESSAUX

M2i

3 juin 2026

1. Introduction
2. BC01 - Construction et alimentation d'une infrastructure de gestion de données
3. BC02 - Analyse exploratoire, descriptive et inférentielle de données
4. BC03 - Analyse prédictive de données structurées par l'intelligence artificielle
5. BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle
6. BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisation des processus de décision
7. BC06 - Direction de projets de gestion de données
8. Conclusion

Introduction du projet

Contexte métier

- La retranscription manuelle d'un audio de guitare vers un fichier MIDI est une tâche chronophage et experte.
- Des solutions existantes cependant la transcription fiable de guitare polyphonique dans des conditions réelles demeure un sujet complexe.
- Besoin d'outils d'assistance à la création musicale et pédagogique

Projet GuitarFlow

- Prototype de transcription automatique d'un audio de guitare vers un fichier MIDI.
- Génération d'un fichier MIDI exploitable en MAO.
- Visualisation piano-roll et interface web de démonstration.

Objectif du POC

- Valider la faisabilité technique d'un pipeline IA audio → MIDI.
- Évaluer les performances du système sur des données réelles.
- Préparer une future phase d'industrialisation.

BC01 - Construction et alimentation d'une infrastructure de gestion de données

Architecture générale du système

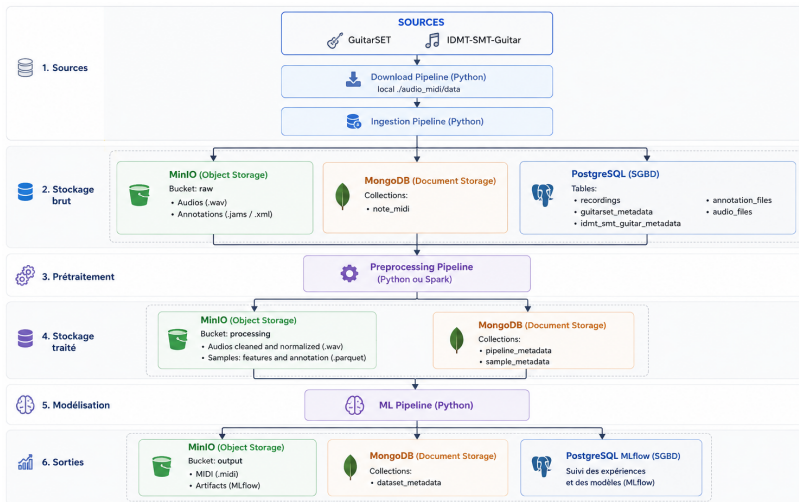
- Infrastructure entièrement containerisée via Docker Compose
- Architecture orientée Machine Learning Platform
- Séparation stockage / traitement / expérimentation

Composants principaux

- **Stockage objet** : MinIO (audio, datasets, artefacts ML)
- **Bases de données** : MongoDB + PostgreSQL
- **Orchestration / ML** : MLflow (tracking expérimental)
- **Traitement distribué** : Apache Spark (cluster master/worker)

Objectif : assurer reproductibilité, scalabilité et traçabilité des données

Infrastructure conceptualisée



Sources de données

- GuitarSet : enregistrements audio de guitare + annotations MIDI.
- IDMT-SMT-Guitar : 4 datasets audio annoté orienté transcription musicale.

Nature des données collectées

- Fichiers audio (WAV).
- Fichiers d'annotation (JAMS, XML, CSV, TXT).
- Métadonnées associées aux fichiers d'annotations.

Stockage locale

- Téléchargement locale des archives (ZIP).
- Extraction locale des archives.

Pipeline d'ingestion

- Upload des datasets vers MinIO (bucket *raw*)
- Extraction des annotations MIDI dans MongoDB (collection *note_midi*)
- Extraction des métadonnées dans PostgreSQL

Sortie du bloc

- Données prêtes pour analyse exploratoire (Bloc 2)

BC02 - Analyse exploratoire, descriptive et inférentielle de données

Présentation des datasets

Objectif de l'analyse

- Étudier la structure et la qualité des données audio de guitare
- Évaluer leur pertinence pour une tâche de transcription automatique vers MIDI

Datasets utilisés

- **GuitarSet [17]**
 - Enregistrements de guitare solo (audio + annotations)
 - Données alignées temporellement (value, time, duration)
 - Large couverture de styles de jeu
- **IDMT-SMT-Guitar [9]**
 - Dataset orienté reconnaissance musicale
 - Contient audio et annotations MIDI associées
 - Variété de conditions d'enregistrement

Enjeu métier

- Vérifier la compatibilité des données avec un pipeline de transcription audio → MIDI

Analyse du dataset GuitarSet

Objectif de l'analyse

- Comprendre la structure et la richesse du dataset
- Évaluer sa pertinence pour la transcription audio → MIDI

Axes d'analyse

- Structure des données (audio / annotations / alignement)
- Analyse musicale descriptive (pitch, densité, accords)
- Analyse du signal audio (spectrogrammes, dynamique)
- Qualité des annotations temporelles

Conclusion intermédiaire

- Dataset riche mais partiellement contraint
- Bonne base pour apprentissage supervisé
- Complément nécessaire avec IDMT-SMT-Guitar

Analyse du dataset IDMT-SMT-Guitar

BC03 - Analyse prédictive de données structurées par l'intelligence artificielle

Modèle non-supervisé

BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle

Architecture du modèle

Choix des hyperparamètres

Pipeline d'entraînement

BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisisation des processus de décision

BC06 - Direction de projets de gestion de données

Présentation du projet

GuitarFlow est une entreprise développant un prototype de service d'intelligence artificielle destiné à la transcription automatique d'enregistrements audio de guitare en fichiers MIDI exploitables en MAO (Musique Assistée par Ordinateur).

Constat métier

La retranscription manuelle d'une performance de guitare vers un format éditable constitue une tâche **chronophage**, nécessitant une **expertise musicale avancée** et difficilement **industrialisation-compatible** pour des usages pédagogiques ou créatifs.

Opportunité

Le projet vise à :

- réduire le temps de transcription audio → MIDI,
- faciliter la production de contenus pédagogiques,
- accélérer les workflows de composition musicale,
- proposer un démonstrateur d'IA appliquée au signal audio.

Objectif principal

Développer un prototype de service capable de :

- recevoir un fichier audio guitare,
- générer automatiquement un fichier MIDI exploitable,
- produire une visualisation de type piano-roll,
- exposer le modèle via une API web déployée sur Hugging Face.

Fonctionnalités incluses

- Audio guitare mono instrument
- Guitare acoustique et électrique
- Accordage standard (EADGBE)
- Génération MIDI
- Visualisation piano-roll
- API REST d'inférence
- Conteneurisation Docker

Fonctionnalités hors périmètre

- Multi-instruments
- Accordages alternatifs
- Temps réel
- Techniques de jeu avancées
- Application mobile native
- Architecture cloud managée

Analyse des besoins et exigences

Exigences fonctionnelles

- Accepter un enregistrement audio de guitare au format WAV.
- Produire automatiquement une transcription au format MIDI exploitable dans un logiciel de MAO.
- Permettre la visualisation des notes détectées sous forme de piano-roll.
- Exposer le service via une API REST et une interface web.
- Garantir la prise en charge d'enregistrements de guitare acoustique et électrique.
- Permettre l'évaluation et la comparaison de plusieurs approches de transcription.

Exigences non fonctionnelles

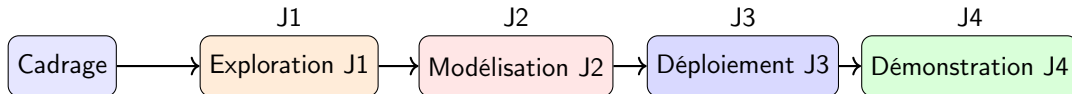
- Temps d'inférence inférieur à 10 secondes pour 1 minute d'audio.
- Reproductibilité complète des expérimentations et des résultats.
- Traçabilité des jeux de données, modèles et métriques.
- Déploiement reproductible par conteneurisation Docker.
- Maintenabilité assurée par une architecture modulaire, des tests automatisés et une documentation technique.

Indicateur	Objectif cible
F1-score de transcription polyphonique	$\geq 90 \%$
Temps moyen d'inférence	< 10 s pour 1 min d'audio
Disponibilité du prototype	Démonstration fonctionnelle avant le 31/07/2026
Déploiement	API accessible via une interface web
Taux de correction manuelle	$< 10 \%$

Approche de pilotage

Le projet a été conduit selon une démarche incrémentale articulée autour de quatre jalons de validation permettant de sécuriser progressivement les différentes phases du POC.

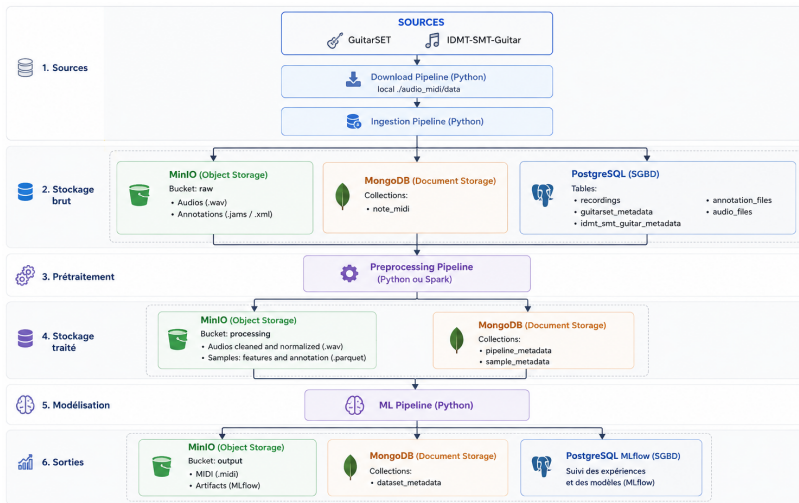
Jalon	Semaine	Critère de validation
J1 – Go Datasets	S4	Données validées et exploitables
J2 – Go Modèle	S6	Baseline opérationnelle
J3 – Go Prototype	S8	Pipeline complet exécutable
J4 – Go Démonstration	S10	API et interface accessibles



[► Voir le Gantt détaillé](#)

Risque	Impact	Prob.	Mitigation
Restrictions de licence sur les data-sets	Élevé	Moyen	Validation juridique des sources avant intégration
Volume de données insuffisant	Élevé	Élevé	Data augmentation + transfer learning
Performances insuffisantes en polyphonie	Élevé	Moyen	Benchmark de plusieurs architectures (baseline / CNN / CRNN)
Temps d'inférence trop élevé	Moyen	Moyen	Optimisation du modèle + quantization
Variabilité de qualité audio	Moyen	Élevé	Normalisation et standardisation du signal audio

Architecture cible



Réalisation du POC

Approche de réalisation

Le POC a été construit selon une logique de pipeline end-to-end couvrant l'ensemble du cycle de vie de la donnée : ingestion, traitement, entraînement et mise à disposition du modèle.

Livrables techniques

- Pipeline d'ingestion des données (GuitarSet, IDMT-SMT-Guitar)
- Pipeline de prétraitement audio et extraction de caractéristiques
- Pipeline d'entraînement et d'évaluation des modèles
- Modèle entraîné et évalué pour la transcription audio → MIDI
- API REST d'inférence (FastAPI)
- Conteneurisation du système via Docker / Docker Compose
- Interface de démonstration déployée sur Hugging Face Spaces

Livrables documentaires

- Note de cadrage et cahier des charges
- Documentation technique des pipelines et de l'architecture
- Guide d'installation et de déploiement
- Support de présentation pour la soutenance

Performance du système

Indicateur	Résultat
F1-score transcription polyphonique	xx %
Temps d'inférence moyen	xx s pour 1 min d'audio
Génération de fichier MIDI	Oui
Visualisation piano-roll	Oui
API d'inférence fonctionnelle	Oui

Exemple de sortie du modèle

- Conversion automatique d'un extrait audio en fichier MIDI exploitable
- Reconstruction des notes jouées sur guitare (mono et polyphonique)
- Visualisation synchronisée type piano-roll

Le POC démontre la faisabilité technique et la valeur métier de la transcription audio → MIDI.

Cette slide fait souvent la différence.

Contenu : - Git - Docker - MLflow - Versionning datasets - Reproductibilité

Message clé : Maîtrise du cycle de vie du projet.

Bilan : - Faisabilité validée - Prototype opérationnel

Perspectives : - Industrialisation - API dédiée - Temps réel - Généralisation multi-instruments

Message clé : Projection vers une phase produit.

Conclusion

- Résumé des contributions
- Résultats obtenus
- Perspectives d'amélioration

Je vous remercie pour votre attention.
Avez-vous des questions

Diagramme de Gantt détaillé

Phase	Tâches	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Cadrage	Validation note de cadrage										
Exploration	Ingestion des données										
	Analyse exploratoire des données										
	Preprocessing audio et extraction des features										
	Construction des datasets frame-wise				J1						
Modélisation	Benchmark des modèles candidats						J2				
	Implémentation du pipeline ML complet								J3		
Déploiement	Développement de l'API d'inférence										
	Dockerisation et déploiement sur Hugging Face Spaces										
Démonstration	Présentation phase1										J4



Martín Abadi et al.

Tensorflow : Large-scale machine learning on heterogeneous systems.
2016.

<https://www.tensorflow.org>.



Apache Software Foundation.

Apache arrow, 2016.

<https://arrow.apache.org>.



Aurélien Géron.

Machine Learning avec Scikit-Learn : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2023.



Aurélien Géron.

Deep Learning avec Keras et TensorFlow : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2024.

Références II



Eric J. Humphrey, Justin Salamon, Oriol Nieto, Jon Forsyth, Rachel M. Bittner, and Juan Pablo Bello.

Jams : A json annotated music specification for reproducible mir research.

In *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014.



MinIO Inc.

Minio object storage, 2016.

<https://min.io>.



MongoDB Inc.

Mongodb, 2009.

<https://www.mongodb.com>.

Références III



Christian Kehling, Jakob Abeßer, Christian Dittmar, and Gerald Schuller.

Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters.

In *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx)*, Erlangen, Germany, 2014.



Christian Kehling, Andreas Männchen, and Arndt Eppler.

Idmt-smt-guitar dataset, 2023.



Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto.

librosa : Audio and music signal analysis in python.

Proceedings of the 14th Python in Science Conference, pages 18–25, 2015.



Wes McKinney.

Data structures for statistical computing in python.

Proceedings of the 9th Python in Science Conference, pages 56–61, 2010.

Références IV



Travis E. Oliphant.

Python for scientific computing.

Computing in Science and Engineering, 9(3) :10–20, 2007.



PostgreSQL Global Development Group.

Postgresql, 1996.

<https://www.postgresql.org>.



Colin Raffel and Daniel P. W. Ellis.

Intuitive analysis, creation and manipulation of midi data with *pretty_midi*.

Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR), 2014.



Sebastián Ramírez.

Fastapi, 2018.

<https://fastapi.tiangolo.com>.



Gilbert Saporta.

Probabilités, analyse des données et statistique.

Technip, Paris, 3 edition, 2011.



Qiaoxi Xi, Seth Dittmer, Justin Pauwels, and Mark Sandler.

Guitarset : A dataset for guitar transcription.

In Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR), Paris, France, 2018.



Matei Zaharia, Andrew Chen, Aaron Davidson, et al.

MLflow : An open source platform for the machine learning lifecycle, 2018.

<https://mlflow.org>.